CUDA-Based Discovery of Complex Junctions in HD Map Road Graphs

Merve Nur Zembil
MMI 713 Applied Parallel Programming on GPU
Middle East Technical University
Ankara, Türkiye
Student No: 2376242

***Abstract*—Complex junctions such as freeway interchanges are important structures in HD maps, but they are not always available as explicit map objects. In the HERE HD Live Map data used in this project, junction-related information is distributed over road topology, routing attributes, lane geometry, and functional-class metadata. This paper presents a CUDA-based implementation for detecting complex junction candidates from a sparse link-level road graph. The method classifies ramp links as secondary/interior candidates and high-level functional-class links as primary/boundary candidates, constructs a compressed sparse row (CSR) dual graph, and detects candidate junctions by grouping connected secondary components and their surrounding primary boundaries. A CPU baseline and a CUDA implementation are implemented with the same input representation and detection parameters. Experiments are performed on small real tile configurations, heavier multi-tile configurations, and synthetic stress configurations generated from the largest real graph. The results show that the CPU baseline is faster on small real graphs because CUDA overhead dominates, while the GPU implementation becomes significantly faster on large synthetic graphs. At 86,000 roads and 265,700 directed adjacency entries, the GPU is approximately 9.45 times faster than the CPU. At 172,000 roads and 531,400 directed adjacency entries, the GPU is approximately 22.43 times faster. The code and experiment files are submitted together with this report; the original HD map data are used locally and are not redistributed publicly. The project source code, scripts, and usage instructions are available at `https://github.com/padfoot0319/junction_detector`.**

***Index Terms*—CUDA, GPU graph processing, HD maps, junction detection, CSR graph, road networks, parallel preprocessing**

I. Introduction

High-definition (HD) maps are commonly used in autonomous driving and intelligent transportation systems because they store detailed lane, topology, and semantic information. However, complex road structures are not always represented as single high-level objects. A freeway interchange, for example, may appear only as several connected road links with different semantic attributes. Therefore, a preprocessing step is needed to infer candidate complex junctions from the underlying road graph.

This project focuses on the parallel implementation of such a preprocessing step. The objective is not to solve all HD map interpretation problems, but to study whether a link-level complex-junction discovery algorithm can benefit from CUDA acceleration. The input is a set of HERE HD Live Map tile files. The preprocessing stage extracts road-link topology and semantic attributes, classifies links into secondary/interior and primary/boundary classes, and exports a compact graph input. The CPU and GPU programs then run on this common intermediate graph representation.

The motivation for using CUDA is that the detection problem becomes a graph traversal and grouping problem over many road links. On small maps, the CPU can process the graph very quickly. However, when larger tile groups or repeated benchmark graphs are used, the number of road links, adjacency entries, and candidate junctions increases. This makes the problem suitable for evaluating GPU graph processing, especially the crossover point where parallelism compensates for CUDA launch, synchronization, and memory-transfer overheads.

The main contributions of this project are:

- a complete CPU and CUDA implementation of a link-level complex-junction detector,
- a preprocessing pipeline that converts HERE HD map data into a compact CSR graph input,
- real, heavy, and synthetic benchmark configurations for different input sizes,
- an experimental comparison showing when the GPU implementation becomes faster than the CPU baseline,
- a MATLAB visualization pipeline for inspecting the detected candidates on real configurations.

II. Problem Statement and Background

The goal is to detect candidate complex junctions from a sparse road-link graph. A road link is represented as a graph vertex. Two vertices are connected if the corresponding road links are adjacent in the topology. Each vertex also has a semantic class derived from the HD map routing attributes. In this implementation, ramp links are treated as secondary or interior candidates, while non-ramp links with functional classes FC_1, FC_2, or FC_3 are treated as primary or boundary candidates. Links outside these classes are ignored by the first version of the detector.

Let the link graph be

$$G = (V, E), \tag{1}$$

where each vertex $v \in V$ represents a road link and each edge $(u, v) \in E$ represents a topological connection between two road links. Each vertex has a class

$$L(v) \in \{\text{other}, \text{primary}, \text{secondary}\}. \tag{2}$$

A detected junction candidate is represented as a pair

$$J_i = (S_i, P_i), \tag{3}$$

where S_i is a set of connected secondary links and P_i is the set of primary links that touch this secondary component. A candidate is accepted only if it satisfies minimum size conditions on the number of secondary links and primary boundary links.

This definition is inspired by the complex-junction detection idea of Yang *et al.* [1], where junctions are found by expanding from secondary roads and collecting primary roads that bound the region. In this project, the full stroke construction and name-based road classification are replaced with a simpler HD map attribute-based classification.

III. Related Work

Yang *et al.* proposed a method for identifying complex junctions in a road network by using a dual graph representation and by distinguishing primary and secondary roads [1]. Their method provides the main conceptual basis for this project. However, their classification relies on road-network concepts such as strokes, road names, and geometric continuity. The present implementation uses HD map routing attributes instead, because these attributes are already available in the HERE HD Live map data.

The CUDA implementation is related to GPU graph traversal methods such as the work of Merrill, Garland, and Grimshaw on scalable GPU breadth-first search (BFS) [2]. Their work motivates the use of CSR storage and frontier-style parallel processing for sparse graphs. In this project, the graph traversal is controlled by semantic link classes, and the final implementation uses parallel label propagation over secondary links rather than a direct block-per-seed BFS. This choice was made after implementation because the final detector groups connected secondary components and because many ramp seeds may belong to the same physical junction.

General CUDA programming principles, such as minimizing unnecessary host-device transfers and using structure-of-arrays layouts, are also followed [3]. However, the road graph is irregular, so perfect memory coalescing is not expected. The main optimization goal is to expose enough independent graph work to the GPU while keeping the memory representation compact.

IV. Method

A. Overall Pipeline

The project is divided into four main stages. First, Python preprocessing reads the HD map tile files and builds the link-level dual graph. Second, the graph is exported to an intermediate JSON file containing CSR arrays and plotting geometry. Third, the CPU and GPU detectors read the same intermediate graph and produce candidate junctions. Finally, MATLAB visualizes the detected candidates for real configurations.

B. Preprocessing and Graph Export

The exporter, implemented in the python script `export_graph_input.py`, reads the configuration file for a selected experiment. Each configuration lists geometry, topology, attribute, road-reference, and routing-attribute files. The exporter builds the road-link graph, queries routing attributes, classifies links, and writes a compact input file under `intermediate/graph_input_configN.json`.

The exported graph contains the following main arrays:

- `link_ids`: original road-link identifiers,
- `row_offsets`: CSR row-offset array,
- `col_indices`: CSR adjacency array,
- `link_type`: semantic class of each link,
- geometry arrays used for distance checks and MATLAB visualization.

The CSR format is used because the road graph is sparse. For a vertex v, its neighbours are stored in the range

$$\texttt{col\_indices}[\texttt{row\_offsets}[v] : \texttt{row\_offsets}[v+1]]. \tag{4}$$

This allows both CPU and GPU implementations to process the same graph structure.

C. CPU Baseline

The CPU implementation is used as the reference for correctness and performance comparison. It uses the same graph input and the same parameters as the CUDA implementation. The CPU version groups connected secondary links using graph traversal. For each secondary component, neighbouring primary links are collected as boundary links. Candidate junctions with too few secondary links or too few primary boundary links are removed.

After initial detection, a candidate-merge step is applied. Candidates are merged if they share at least one primary boundary link. They may also be merged when their secondary regions are closer than 50 m or when their centroids are within 300 m. These distance thresholds were selected heuristically by inspecting the output visualizations.

D. CUDA Implementation

The CUDA implementation uses the same CSR graph representation. The main device-side work is divided into two stages. The first stage labels connected components of secondary links. The second stage evaluates candidate merging and centroid-based proximity on the GPU.

In the final version, direct seed-by-seed BFS from the Phase II design was replaced with parallel label propagation over the secondary subgraph. Initially, each secondary vertex receives its own label and every non-secondary vertex receives label -1. Then each iteration lets a secondary vertex inspect neighbouring secondary vertices and adopt the smallest label it sees. This process is repeated until no label changes occur.

The simplified label-propagation logic is shown below:

```
Initialize label[v] = v for secondary links, otherwise −1
repeat
    changed = false
    for all secondary links v in parallel do
        best = label[v]
        for all neighbours u of v do
            if u is secondary and label[u] < best then
                best = label[u]
            end if
        end for
        if best < label[v] then
            label[v] = best; changed = true
        end if
    end for
until changed is false
```

After labels are copied back to the host, primary boundary sets are built by scanning adjacency around the labeled secondary components. This avoids allocating a dense $N \times N$ boundary matrix. The dense approach was an unfruitful implementation path because it becomes impossible for synthetic graphs with tens of thousands of road links. Replacing it with adjacency-based boundary extraction reduces the memory requirement from $O(|V|^2)$ to approximately $O(|E|)$.

Candidate centroids and pairwise merge decisions are then processed in the CUDA merge stage. The merge stage compares candidate pairs using shared primary boundary information and geometric proximity. For synthetic configurations, the detector prints timing rows but skips large JSON visualization outputs, because the synthetic maps are too large to inspect visually.

V. Comparison with Phase II Design

The Phase II design proposed a host-device pipeline where preprocessing and result extraction run on the host, while junction discovery and optional cleanup run on the GPU. This structure was kept in the final implementation. The input graph is still built on the host, stored in CSR format, copied to the GPU, processed in CUDA kernels, and converted back into candidate junction results.

The memory design from Phase II also remained mostly valid. CSR arrays are used for the graph, and link classes are stored in a structure-of-arrays style. This matches the original plan to use `row_offsets`, `col_indices`, and `link_type` arrays as the main device representation.

The main algorithmic change is the replacement of block-per-seed BFS with parallel secondary-component labeling. In Phase II, each ramp seed was expected to launch an expansion over a frontier. During implementation, many ramp links were found to belong to the same physical junction. Processing every seed independently would create duplicate regions and extra conflict handling. Therefore, the final CUDA version labels connected secondary regions directly. This still follows the same high-level idea of expanding through secondary links until primary boundaries are reached, but it expresses the problem as connected-component labeling rather than many independent BFS runs.

Another change is that the planned P-expansion validation step was disabled in the final tuned version. In experiments, primary expansion tended to over-expand some candidates and merge nearby structures too aggressively. Therefore, `P_EXPAND_MAX_ROUNDS` was set to zero, and candidate merging was controlled using shared boundary links and geometric distance thresholds. This made the result more stable for the selected data.

VI. Experimental Setup

A. Configurations

Ten configurations were used in the final benchmark. Configurations 1–3 are smaller real map configurations. Configurations 4–7 are heavier real or combined configurations generated from available HERE HD map tile files. Configurations 8–10 are synthetic stress configurations generated from configuration 7. The synthetic cases repeat the graph while shifting the geometry and link IDs so that the workload increases without changing the underlying local junction pattern.

The synthetic configurations are not used to claim real-world map diversity. Their purpose is to study scalability and to identify the point where the GPU has enough work to overcome its overhead. MATLAB visualization is generated only for real configurations, while synthetic configurations are used only for timing.

B. Measurement

The reported `Algorithm_s` value measures the detector and merge stage inside the CPU or GPU executable. It excludes Python preprocessing and MATLAB visualization. It also excludes output writing because timing starts after graph input loading and ends before JSON export. For the GPU implementation, the measured interval includes CUDA allocations, transfers, kernel execution, synchronization, and result construction within the detector stage.

The main evaluation metric is CPU-to-GPU speedup:

$$\text{Speedup} = \frac{T_{CPU}}{T_{GPU}}. \tag{5}$$

Values larger than 1 indicate that the GPU is faster.

VII. Results and Discussion

The final experiment confirms the expected crossover behaviour. For small and medium real graphs, the CPU implementation is faster. For example, configuration 7 has 1,720 roads and 5,314 directed adjacency entries. The CPU completes this case in 0.003257 s, while the GPU takes 0.206078 s. At this size, the graph is too small to amortize CUDA overhead.

The synthetic stress tests show the opposite trend. Configuration 8 has 17,200 roads and 53,140 directed adjacency entries. The CPU still remains slightly faster, with 0.173575 s compared to 0.280479 s on the GPU. However, at configuration 9, which has 86,000 roads and 265,700 directed adjacency entries, the GPU time is 0.461749 s while the CPU time is 4.361326 s. This corresponds to a speedup of approximately

9.45. At configuration 10, with 172,000 roads and 531,400 directed adjacency entries, the GPU reaches 0.849552 s while the CPU takes 19.057145 s, corresponding to approximately 22.43 times speedup.

Table I is the output table of all configurations.

TABLE I
RUNTIME RESULTS FOR ALL CONFIGURATIONS

Cfg.	Roads	Edges	Junc.	CPU (s)	GPU (s)	Speedup
1	40	132	1	0.000010	1.553383	0.000006x
2	78	256	2	0.000162	0.143425	0.001x
3	138	422	4	0.000128	0.133040	0.001x
4	328	1,056	10	0.000531	0.154187	0.003x
5	755	2,492	17	0.001306	0.208367	0.006x
6	1,470	4,698	34	0.002999	0.232962	0.013x
7	1,720	5,314	35	0.003257	0.206078	0.016x
8	17,200	53,140	350	0.173575	0.280479	0.619x
9	86,000	265,700	1,750	4.361326	0.461749	9.45x
10	172,000	531,400	3,500	19.057145	0.849552	22.43x

The results are consistent with the design expectations. CUDA is not automatically faster for every input size. The first GPU run is especially slow because it includes CUDA context initialization and other one-time overheads. Even after the first run, small road graphs contain only a few hundred or a few thousand edges, which is too little work for a GPU. Once the graph grows to hundreds of thousands of adjacency entries, the available parallelism becomes large enough and the GPU becomes clearly faster.

VIII. CORRECTNESS AND VISUALIZATION

For real configurations, both CPU and GPU outputs are written as JSON files under the `exports` directory. The MATLAB script `run_road_network.m` loads these outputs and creates PNG visualizations under the `visualizations` directory. The visualization shows non-junction roads as background and highlights the detected primary/boundary and secondary/interior links. Some representative CPU and GPU outputs are shown in Fig. 1. These examples are useful for checking whether the detected candidates correspond to plausible interchange regions and also show that the CPU and GPU implementations produce the same visual output for the same input configuration.

The CPU and GPU versions use the same input graph, same classification, and same merge thresholds. Therefore, the number of detected junctions should match for each configuration. In the final benchmark, CPU and GPU junction counts match for all configurations. This is an important correctness check before interpreting the runtime comparison.

IX. LIMITATIONS

The first limitation is the simplified road classification. The method uses `is_ramp` as the main secondary indicator and functional classes FC_1, FC_2, and FC_3 as primary indicators. This works reasonably for freeway interchange examples, but it is not a complete semantic model of all complex junction types. Roundabouts and urban intersections may need additional attributes or geometric rules.

The second limitation is that the largest real graph is still small from a GPU perspective. The real or combined configurations contain up to 1,720 roads and 5,314 directed adjacency entries. This is useful for correctness and visualization, but not enough to demonstrate GPU speedup. Synthetic benchmarks were therefore needed to evaluate scalability.

The third limitation is that the synthetic configurations repeat the same base graph. They are useful for stress testing, but they do not represent independent real-world map diversity. They should be interpreted as scalability experiments rather than as additional real map cases.

Finally, the current GPU implementation still includes host-side result construction after the label propagation stage. More of this work could be moved to the GPU in a future version, especially candidate grouping and sparse boundary extraction.

X. CONCLUSION AND FUTURE WORK

This project implemented a complete CPU and CUDA pipeline for detecting complex junction candidates in HD map road graphs. The final implementation follows the original Phase II design in its use of host preprocessing, CSR graph storage, CUDA graph processing, and CPU-GPU benchmarking. The main design change was replacing seed-level BFS with parallel secondary-component label propagation, which better matched the final graph formulation and reduced duplicate seed handling.

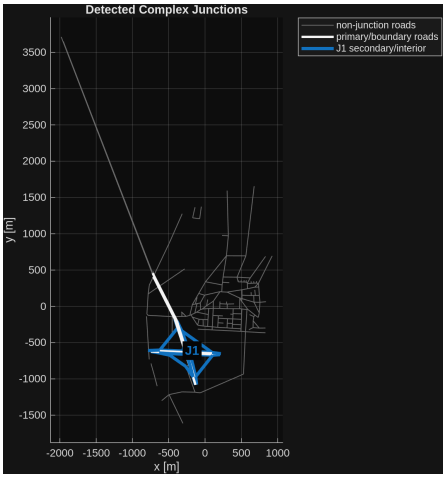
The experiments show that the CPU implementation is faster on small real HD map graphs because CUDA overhead dominates. However, synthetic scalability experiments show that the GPU implementation becomes much faster when the graph size grows. The GPU achieves approximately 9.45 times speedup at 86,000 roads and approximately 22.43 times speedup at 172,000 roads.

Future work should improve the semantic classification of primary and secondary roads, test the algorithm on larger real HD map regions, move more candidate extraction work to the GPU, and compare the detected candidates against manually labelled reference junctions. Additional CUDA profiling could also be used to separate kernel time, memory-copy time, and host-side postprocessing time.

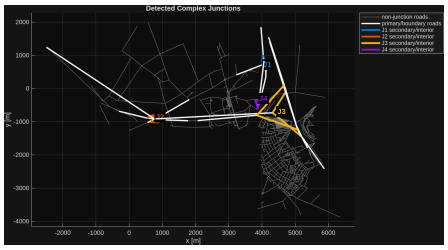
APPENDIX A
IMPLEMENTATION NOTES

The project is organized around a small number of scripts and executables. The Python exporter creates graph inputs, the C++ executable runs the CPU baseline, the CUDA executable runs the GPU version, and the shell script builds and executes all experiments. The MATLAB script is used only for visualization of real configurations.

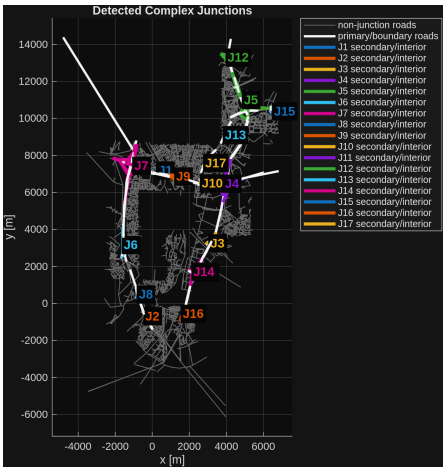
The build script compiles the project using CMake, exports graph inputs for real configurations, skips preprocessing for synthetic configurations, runs CPU and GPU detectors, and visualizes only the non-synthetic outputs. Synthetic configurations are skipped during visualization because their images are too large to inspect and would slow down the experiment.



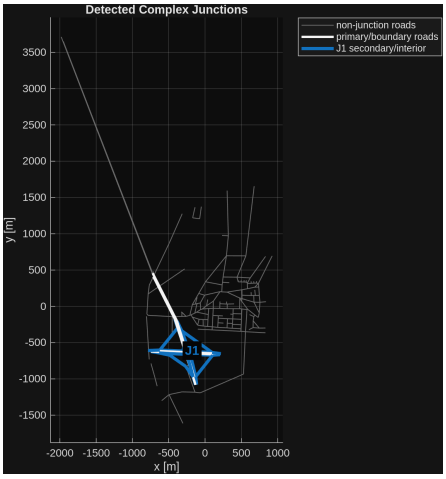
CPU, config 1: 1 junction



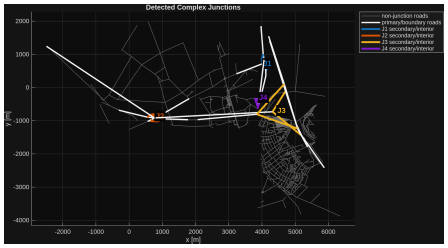
CPU, config 3: 4 junctions



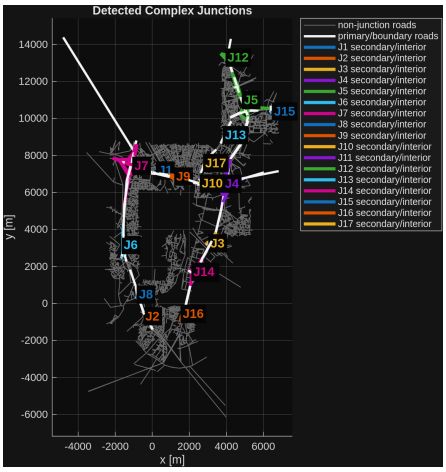
CPU, config 5: 17 junctions



GPU, config 1: 1 junction



GPU, config 3: 4 junctions



GPU, config 5: 17 junctions

Fig. 1. Representative visual outputs for real configurations. The first row shows CPU outputs and the second row shows GPU outputs for the same configurations. The matching figures indicate that both implementations produce the same detected junction candidates.

APPENDIX B
REPRODUCIBILITY SUMMARY

The main command used for the final benchmark is:

```
./build_and_run.sh
```

Before creating synthetic inputs, the base real graph must be exported (this step is also described in the project's README). The synthetic generator then duplicates the selected base graph with repeat counts of 10, 50, and 100. In the final configuration numbering, these correspond to configurations 8, 9, and 10.

REFERENCES

[1] J. Yang, K. Zhao, M. Li, Z. Xu, and Z. Li, "Identifying Complex Junctions in a Road Network," *ISPRS International Journal of Geo-Information*, vol. 10, no. 4, 2021.

[2] D. Merrill, M. Garland, and A. Grimshaw, "High Performance and Scalable GPU Graph Traversal," Technical Report CS-2011-05, University of Virginia, 2011.

[3] NVIDIA, *CUDA C++ Programming Guide*. NVIDIA Corporation.